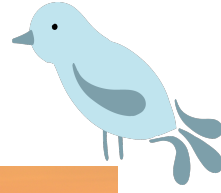


Bit Manipulation



Good
Morning



Content

01. Single number 1
02. Single number 2
03. Single number 3
04. Maximum & pair
05. Count of pairs with maximum AND

Q1. Given N elements, every element repeats twice except one. find the unique element.

$$ar[7] = \{ \underset{0}{3}, \underset{1}{2}, \underset{2}{3}, \underset{3}{7}, \underset{4}{2}, \underset{5}{8}, \underset{6}{7} \} \quad Ans = 8$$

$$ar[] = \{ 4, 5, 5, 4, 1, 6, 6 \} \quad Ans = 1$$

* value of $120 \wedge 5 \wedge 6 \wedge 6 \wedge 120 \wedge 5 \rightarrow \underline{\underline{0}}$

property $A \wedge A = 0$

Q1. XOR of entire array

$$ans = 0$$

```
for(i=0; i<n; i++){  
    |  
    ans = ans ^ ar[i];  
    |  
    3
```

return ans;

$$Tc: O(n)$$

$$Sc: O(1)$$

* Approach 2

→ Try to analyse bit operations bit by bit.

arr[] = { 2, 3, 5, 6, 3, 6, 2 }

2	→	0	1	0
3	→	0	1	1
5	→	1	0	1
6	→	1	1	0
3	→	0	1	1
6	→	1	1	0
2	→	0	1	0
		3	6	3
		↓	↓	↓
		$2x+1$	$2x$	$2x+1$
		1	0	1

Every element repeats twice except one.

Count the set bits at each position

→ $2x$ (even)

→ $2x+1$ (odd)

Idea →

01. Iterate on all bits one by one
02. Count no. of ele where 'i' bit is set
03. If $\text{count} \% 2 == 1 \rightarrow$ set 'i' bit in your ans.

*

ans = 0

for (i = 0; i < 32; i++) {

int count = 0

for (j = 0; j < n; j++) {

if (arr[j] & (1 << i) > 0) count++;

if (count & 1 == 1) // count is odd

ans = ans | (1 << i);

}

TC: $O(n)$

SC: $O(1)$

* Single number 2

Given an arr[N], all the elements occurs thrice except one, Find the unique element
↓
occurring once

arr[] = { 2, 3, 2, 2, 4, 3, 3 } Ans = 4

arr[] = { 4, 5, 5, 4, 1, 6, 6, 4, 5, 6 } Ans = 1

01. Brute force

For each & every ele, we have to iterate on entire array & count its frequency

```
for (i = 0 → n)
|
|   count = 0
|
|   for (j = 0 → n)
|   |
|   |   if (arr[i] == arr[j]) count++;
|   |
|   |   3
|   |
|   |   if (count == 1) return arr[i];
|   |
|   3
3
```

TC: $O(n^2)$

SC: $O(1)$

02. HashMap

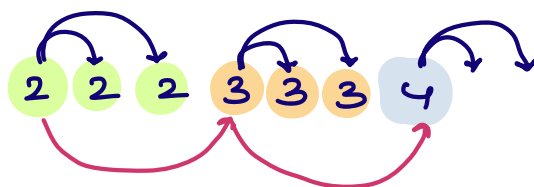
→ Fill your hashmap

→ Iterate on arr again & check frequency of elements using HM

TC: $O(n)$

SC: $O(n)$

03. Sorting



Ans = 4

TC: $O(n \log n)$

SC: depends

upon

sorting algo

arr[13] =

5	7	5	4	7	11	11	9	11	7	5	4	4
---	---	---	---	---	----	----	---	----	---	---	---	---

	3	2	1	0
5 =	0	1	0	1
7 =	0	1	1	1
5 =	0	1	0	1
4 =	0	1	0	0
7 =	0	1	1	1
11 =	1	0	1	1
11 =	1	0	1	1
9 =	1	0	0	1
11 =	1	0	1	1
7 =	0	1	1	1
5 =	0	1	0	1
4 =	0	1	0	0
4 =	0	1	0	0

4 9 6 10

→ Multiple of 3 then 0
else → 1

$4 \% 3 == 1$
 $9 \% 3 == 0$ $6 \% 3 == 0$ $10 \% 3 == 1$

1 0 0 1 Ans = 9

* ans = 0

```
for (i = 0; i < 32; i++) {
```

```
    int count = 0
```

```
    for (j = 0; j < n; j++) {
```

```
        if (ar[j] & (1 << i) > 0) count ++;
```

```
    if (count % 3 == 1) // count is odd
```

```
        ans = ans | (1 << i);
```

```
}
```

Tc: $O(n)$
Sc: $O(1)$

* Extensions

→ Every ele is repeating twice except one, which is repeating twice

Ans ⇒ The exact same code will work

→ Every ele is repeating 4 times, except one ele which is repeating once

Ans = Take XOR of entire array

* One ele is occurring twice

Ans = if $(c \% 4 \neq 0)$ ans $\rightarrow$ set the i bit

* One ele is repeating thrice

Ans $\rightarrow$ XOR of all elements

* Single Number 3

Given N array elements, all the elements will occur twice except 2 unique elements. Find those two unique elements

arr[] = {4, 5, 5, 4, 1, 2} Ans = 1 and 2

arr[] = {1, 2, 3, 1, 2, 4} Ans = 3 and 4

Brute force $\rightarrow$ For each ele, go & count the no. of duplicate elements

if (count == 1) ans = (x)

TC: $O(n^2)$

if (count == 1) ans = (y)

SC: $O(1)$

Idea 2 → Use HashMap

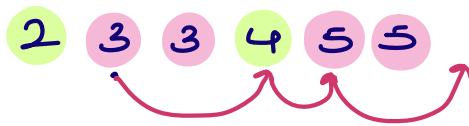
→ store freq of all the ele in HM

→ Iterate & check what all ele have their
freq as 1

TC: $O(n)$

SC: $O(n)$

Idea 3 → Sort the array



TC: $O(n \log n)$

SC: depends upon
sorting algo

Idea 4 XOR of all elements

$ar[] = \{ 3 \ 6 \ 4 \ 4 \ 3 \ 8 \}$

$xor = 3 \wedge 6 \wedge 4 \wedge 4 \wedge 3 \wedge 8$

$xor = 6 \wedge 8$

Obs → xor of all elements = xor of two unique elements

$$\text{arr}[12] =$$

10	8	8	9	12	9	6	11	10	6	12	17
1010	1000	1000	1001	1100	1001	0110	1011	1010	0110	1100	10001

01. Step 1 XOR of all elements

$$\text{xor} = 11 \wedge 17$$

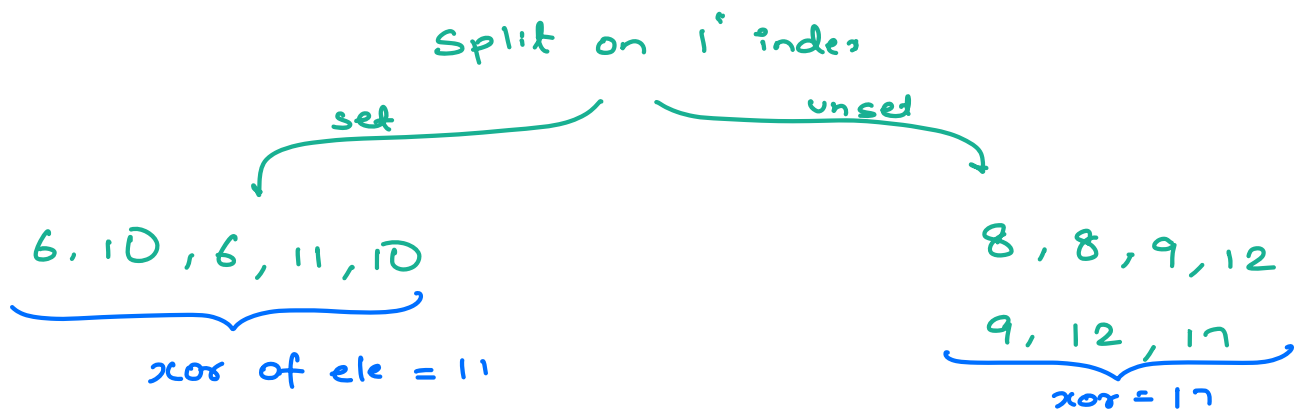
	4	3	2	1	0
11 =	0	1	0	1	1
17 =	1	0	0	0	1
	1	1	0	1	0

At this bit position, both the distinct ele has different bit

02. Split the array on 1st indexed bit

$$\text{arr}[12] =$$

10	8	8	9	12	9	6	11	10	6	12	17
1010	1000	1000	1001	1100	1001	0110	1011	1010	0110	1100	10001



void single3 (int () ar)

01. xor of entire arr

val = 0

for (i = 0; i < n; i++) {

val = val ^ ar[i];

}

// val → xor of two unique ele

02. Identify the set bit in val

int idx = -1

for (i = 0; i < 32; i++) {

if (val & (1 << i) > 0) {

idx = i;

break;

;

}

TC: $O(n)$

SC: $O(1)$

03. Split the entire arr on the basis of this idx

set = 0 unset = 0

for (i = 0; i < n; i++) {

if (ar[i] & (1 << idx) > 0) set = set ^ ar[i];

else unset = unset ^ ar[i];

}

print (set + " " + unset);

$$\text{arr}[12] =$$

10	8	8	9	12	9	6	11	10	6	12	17
<u>1010</u>	<u>1000</u>	<u>1000</u>	<u>1001</u>	<u>1100</u>	<u>1001</u>	<u>0110</u>	<u>1011</u>	<u>1010</u>	<u>0110</u>	<u>1100</u>	<u>10001</u>

$$\text{set} = 10 \wedge 8 \wedge 8 \wedge 9 \wedge 12 \wedge 9 \wedge 11 \wedge 10 \wedge 12 \Rightarrow 11$$

$$\text{unset} = 6 \wedge 6 \wedge 17 \Rightarrow 17$$

$$\underline{8:23 \rightarrow 8:32 \text{ AM}}$$